

Mineração de Dados: Conceitos e Aplicações

Módulo 3: Técnicas de Classificação

Prof. Elton Sarmanho¹
eltonss@ufpa.br

¹Faculdade de Sistemas de Informação - UFPA Campus Cametá

29 de junho de 2026

Roteiro

Introdução à Classificação

Árvores de Decisão

Métodos Probabilísticos

K-Vizinhos Mais Próximos (KNN)

Máquinas de Vetores de Suporte (SVM)

Métodos Ensemble

Avaliação de Modelos

O que é Classificação?

Aprendizado Supervisionado

- ▶ Aprender a partir de **exemplos rotulados**
- ▶ Cada exemplo: atributos + classe
- ▶ Meta: prever a classe de novos exemplos

Definição

Dado um conjunto de treino $(\mathbf{x}_i, y_i)$,
aprender $f(\mathbf{x}) \rightarrow y$

Aplicações:

-  **Spam:** e-mail legítimo ou lixo
-  **Médico:** tumor benigno/maligno
-  **Crédito:** aprovação de empréstimo
-  **Churn:** cliente irá cancelar?
-  **Fraude:** transação suspeita
-  **Imagem:** reconhecimento de objetos

Tipos de Classificação

Tipo	Descrição	Exemplo
Binária	2 classes: $y \in \{0, 1\}$	Spam vs. Não-spam
Multiclasse	$k > 2$ classes	Dígito reconhecido (0–9)
Multirótulo	Múltiplos rótulos multêneos	si- Gêneros de um filme

Nota

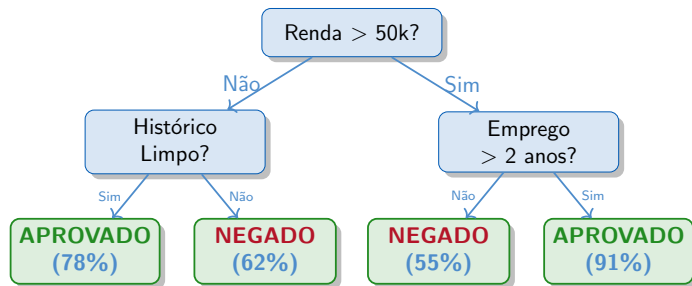
A maioria dos algoritmos clássicos é projetada para **classificação binária**. Extensões para multiclasse: *One-vs-Rest* (OvR) e *One-vs-One* (OvO).

Árvores de Decisão: Conceito

Como funciona?

- ▶ **Nó interno:** teste em um atributo
- ▶ **Ramo:** resultado do teste
- ▶ **Folha:** classe predita

Para classificar: percorra da raiz até uma folha seguindo os atributos do exemplo.



Algoritmos: ID3, C4.5 e CART

Característica	ID3	C4.5	CART
Critério	Entropia/IG	Gain Ratio	Índice Gini
Tipo de divisão	Multiway	Multiway	Binária
Atrib. contínuos	Não	Sim	Sim
Valores ausentes	Não	Sim	Sim
Regressão	Não	Não	Sim
Poda	Não	Pós-poda	Custo-complexidade

Critério de impureza

- ▶ **Entropia:** $H(S) = - \sum_i p_i \log_2 p_i$
- ▶ **Gini:** $G(S) = 1 - \sum_i p_i^2$

Poda (Pruning)

Evita **overfitting**: limita profundidade, mínimo de amostras por folha, ou remove ramos após a construção.

Propriedades das Árvores de Decisão

Vantagens

- ▶ **Interpretabilidade:** regras legíveis por humanos
- ▶ **Não-paramétrico:** sem hipóteses sobre a distribuição
- ▶ **Sem normalização:** não precisa escalar features
- ▶ **Tipos mistos:** numérico e categórico nativamente

Limitações

- ▶ **Instabilidade:** pequenas mudanças nos dados $\Rightarrow$ árvore diferente
- ▶ **Overfitting:** árvores profundas memorizam o treino
- ▶ Fronteiras de decisão apenas horizontais/verticais

Solução

Métodos Ensemble (Random Forest, Boosting) combinam múltiplas árvores para

Naive Bayes

Teorema de Bayes

$$P(c | \mathbf{x}) = \frac{P(\mathbf{x} | c) P(c)}{P(\mathbf{x})}$$

- ▶ $P(c)$: probabilidade *a priori*
- ▶ $P(\mathbf{x}|c)$: verossimilhança
- ▶ $P(c|\mathbf{x})$: probabilidade *a posteriori*

Hipótese “ingênua”: atributos são condicionalmente independentes dada a classe.

Variantes:

Variante	Distribuição	Uso
Gaussiano	Normal	Features contínuas
Multinomial	Multinomial	Texto (freq. palavras)
Bernoulli	Bernoulli	Features binárias

Suavização de Laplace

Evita probabilidade zero para valores não vistos no treino: adiciona $\alpha = 1$ às

Regressão Logística

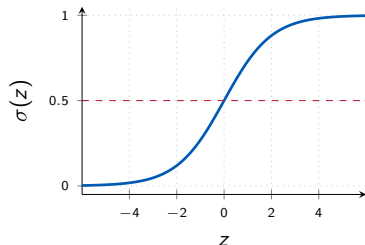
Função Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

onde $z = \mathbf{w}^T \mathbf{x} + b$

- ▶ Saída em $(0, 1)$: probabilidade
- ▶ Limiar = 0.5: classe positiva se $\sigma > 0.5$

Multiclasse: usa função **Softmax** para k classes.



Quando usar?

Classificação **binária linear** com necessidade de probabilidades calibradas.

KNN: K-Vizinhos Mais Próximos

Como funciona?

1. Dado novo exemplo $\mathbf{x}$
2. Calcule a distância para **todos** os exemplos de treino
3. Encontre os k mais próximos
4. **Vote** pela classe majoritária

Distância mais comum: Euclidiana

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_i (a_i - b_i)^2}$$

Efeito de k

- ▶ $k = 1$: fronteira irregular
Overfitting (muito sensível ao ruído)
- ▶ $k = n$: fronteira suave
Underfitting (ignora estrutura)
- ▶ k ótimo via **validação cruzada**
(ponto de partida: $k \approx \sqrt{n}$)

Atenção

Requer **normalização** das features!

Distâncias são sensíveis à escala.

SVM: Margem Máxima

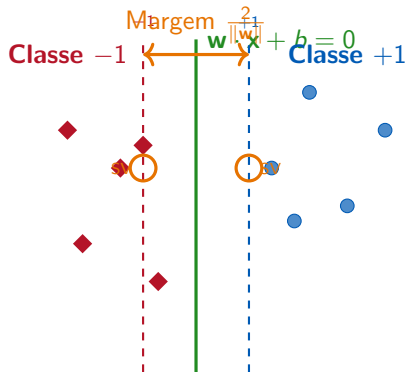
Conceito

Encontrar o hiperplano que **maximiza a margem** entre as classes.

Vetores de Suporte: os exemplos mais próximos do hiperplano, sendo os únicos que determinam a fronteira.

Soft Margin

Parâmetro C controla o trade-off entre margem e erros de treino



SVM: Kernel Trick

Por que kernels?

Quando os dados **não são linearmente separáveis**, o kernel mapeia para um espaço de maior dimensão *sem calcular o mapeamento explicitamente*.

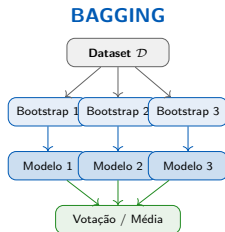
Kernel	Fórmula	Quando usar
Linear	$K(\mathbf{x}, \mathbf{z}) = \mathbf{x} \cdot \mathbf{z}$	Dados linearmente separáveis, texto
Polinomial	$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x} \cdot \mathbf{z} + c)^d$	Interações de grau d
RBF (Gaussiano)	$K(\mathbf{x}, \mathbf{z}) = e^{-\gamma \ \mathbf{x} - \mathbf{z}\ ^2}$	Fronteiras complexas (padrão)

Métodos Ensemble: Visão Geral

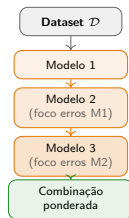
Princípio

Muitos modelos simples > um modelo complexo

- ▶ **Bagging:** paralelo, reduz *variância*
- ▶ **Boosting:** sequencial, reduz *viés*



BOOSTING



Random Forest

Como funciona?

1. **Bootstrap:** criar T amostras com reposição
2. **Árvore aleatória:** em cada divisão, avaliar apenas $m \approx \sqrt{p}$ features aleatórias
3. **Votação:** predição final = classe majoritária das T árvores

A aleatoriedade nas features **descorrelaciona** as árvores, reduzindo a variância.

Vantagens

- ▶ Muito robusto a overfitting
- ▶ Lida bem com muitas features
- ▶ **Importância de features** (MDI, MDA)
- ▶ Poucas decisões de hiperparâmetros

Limitação

Menos interpretável que uma única árvore.
Para explicabilidade: usar **SHAP**.

XGBoost, LightGBM e CatBoost

Gradient Boosted Decision Trees (GBDT)

Constroem árvores **sequencialmente**, cada uma corrigindo os erros da anterior.
Dominam benchmarks em dados tabulares.

Algoritmo	Inovação chave	Destaque
XGBoost	Regularização L1/L2	Competições Kaggle desde 2015
LightGBM	GOSS + crescimento leaf-wise	~10× mais rápido em dados grandes
CatBoost	Ordered target encoding	Suporte nativo a categóricas

Matriz de Confusão

O que é?

Tabela que organiza as predições vs. rótulos reais. Revela **tipos de erros** cometidos pelo modelo.

- ▶ **VP**: acertou positivo
- ▶ **VN**: acertou negativo
- ▶ **FP**: disse positivo, era negativo
- ▶ **FN**: disse negativo, era positivo

	Previsto +	Previsto -
Real +	VP (Verdadeiro Positivo)	FN (Falso Negativo)
Real -	FP (Falso Positivo)	VN (Verdadeiro Negativo)

Métricas de Avaliação

Fórmulas

$$\text{Acurácia} = \frac{VP + VN}{VP + VN + FP + FN}$$

$$\text{Precisão} = \frac{VP}{VP + FP}$$

$$\text{Recall} = \frac{VP}{VP + FN}$$

$$\text{F1-Score} = 2 \cdot \frac{P \cdot R}{P + R}$$

Exemplo: Detecção de Doença

Imagine 100 pacientes (10 doentes, 90 saudáveis):

- ▶ Modelo detectou 8 dos 10 doentes (VP=8)
- ▶ 5 saudáveis foram ditos doentes (FP=5)
- ▶ **Precisão** = $8/13 = 0,62$
- ▶ **Recall** = $8/10 = 0,80$
- ▶ **F1** = 0,70

Quando Usar Cada Métrica?

Situação	Métrica	Motivo
Classes balanceadas	Acurácia	Simples e intuitiva
FP tem alto custo	Precisão	Spam: não bloquear e-mail legítimo
FN tem alto custo	Recall	Diagnóstico: não perder paciente doente
Equilíbrio P/R	F1-Score	Geral para dados desbalanceados
Threshold incerto	AUC-ROC	Avalia ranking de probabilidades
Alta desbalanceamento	AUC-PR	Fraude, doenças raras

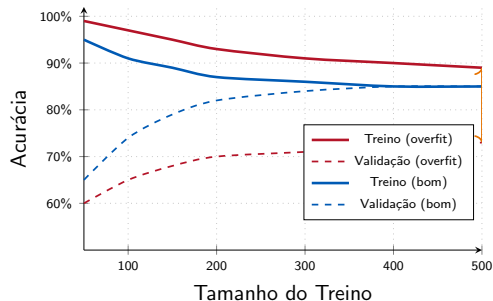
Curva de Aprendizado: O que o Modelo Aprende?

O que é?

Plota o desempenho (acurácia) em função do **tamanho do conjunto de treino**.

Avalia se o modelo está **aprendendo de fato** ou apenas memorizando dados.

- ▶ **Curva de treino:** desempenho nos dados de treino
- ▶ **Curva de validação:** desempenho em dados novos



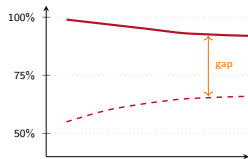
Diagnóstico pelo gap

O **gap** entre as duas curvas revela se há overfitting, underfitting ou aprendizado saudável.

Curva de Aprendizado: Três Cenários

Overfitting

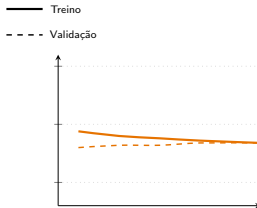
(Alta Variância)



Treino ótimo, validação baixa
Solução: mais dados, regularização,
menos features

Underfitting

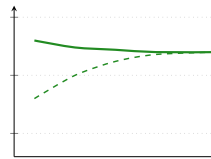
(Alto Viés)



Ambas as curvas baixas, gap pequeno
Solução: modelo mais complexo,
mais features, menos regularização

Modelo Ideal

(Balanceado)



Curvas convergem em valor alto
Gap pequeno ao final
Generaliza bem!

O que é a Curva ROC?

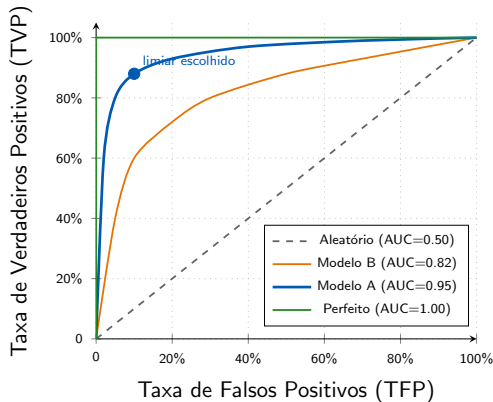
Plota a **Taxa de Verdadeiros Positivos** (Recall/Sensibilidade) contra a **Taxa de Falsos Positivos** para *todos os possíveis limiares* de decisão.

$$\text{▶ } TVP = \frac{VP}{VP + FN} \quad (\text{eixo Y})$$

$$\text{▶ } TFP = \frac{FP}{FP + VN} \quad (\text{eixo X})$$

AUC: Área Sob a Curva

- ▶ AUC = 1.0: modelo **perfeito**
- ▶ AUC = 0.5: modelo **aleatório**
- ▶ AUC < 0.5: pior que o acaso



Cenário

Classificador atribui probabilidade de spam a 10 e-mails (5 spam, 5 legítimos). Variando o limiar, calculamos os pontos da curva ROC.

Limiar	VP	FP	TVP	TFP
0.9	2	0	0.40	0.00
0.7	3	1	0.60	0.20
0.5	4	1	0.80	0.20
0.3	5	2	1.00	0.40
0.1	5	5	1.00	1.00

Limiar baixo: captura mais spam (TVP↑), mas bloqueia mais legítimos (TFP↑).

O ponto azul na tabela (limiar 0.5) equilibra os erros.

AUC-ROC vs AUC-PR

- ▶ **AUC-ROC:** ideal para classes **balanceadas**; avalia separação geral entre classes
- ▶ **AUC-PR** (Precisão × Recall): preferível em dados **muito desbalanceados** (fraude, doenças raras)

Referência rápida

AUC $\geq$ 0.90: excelente

AUC $\geq$ 0.80: bom

AUC $\geq$ 0.70: aceitável

AUC $<$ 0.60: rever o modelo

ROC Spam: Passo a Passo — Como Calcular TVP e TFP

10 e-mails classificados (5 spam ●, 5 legítimos ○)

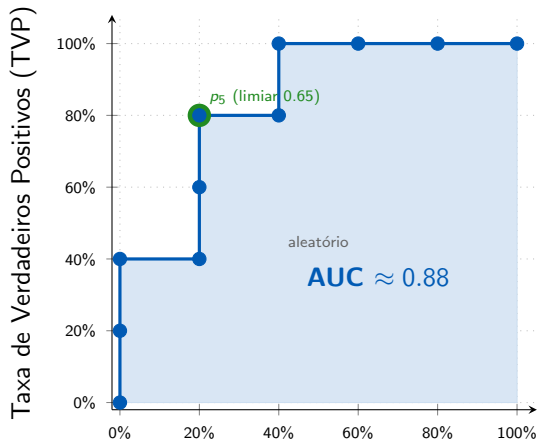
O modelo atribui uma probabilidade de spam a cada e-mail. Ordenamos do maior ao menor:

E-mail	P(spam)	Rótulo Real	VP acum.	FP acum.	TVP / TFP
e_1	0.95	Spam	1	0	0.20 / 0.00
e_2	0.88	Spam	2	0	0.40 / 0.00
e_3	0.80	Legítimo	2	1	0.40 / 0.20
e_4	0.75	Spam	3	1	0.60 / 0.20
e_5	0.65	Spam	4	1	0.80 / 0.20
e_6	0.55	Legítimo	4	2	0.80 / 0.40
e_7	0.40	Spam	5	2	1.00 / 0.40
e_8	0.30	Legítimo	5	3	1.00 / 0.60
e_9	0.20	Legítimo	5	4	1.00 / 0.80
e_{10}	0.10	Legítimo	5	5	1.00 / 1.00

ROC Spam: Curva Plotada com os 10 Pontos

Pontos da Curva

Pt	Limiar	TFP	TVP
p_0	—	0.00	0.00
p_1	0.95	0.00	0.20
p_2	0.88	0.00	0.40
p_3	0.80	0.20	0.40
p_4	0.75	0.20	0.60
p_5	0.65	0.20	0.80
p_6	0.55	0.40	0.80
p_7	0.40	0.40	1.00
p_8	0.30	0.60	1.00
p_9	0.20	0.80	1.00
p_{10}	0.10	1.00	1.00



👍 Benefícios da AUC-ROC

- ▶ **Independente do limiar:** avalia o modelo para *todos* os limiares de decisão simultaneamente, sem fixar um valor de corte
- ▶ **Invariante à escala:** não depende dos valores absolutos das probabilidades previstas, apenas da ordenação
- ▶ **Robusta a classes balanceadas:** compara modelos sem viés causado pela proporção entre classes
- ▶ **Comparação direta:** um número único (0 a 1) para comparar modelos diferentes de forma objetiva
- ▶ **Interpretação probabilística:** $AUC =$ probabilidade de o modelo ranquear um positivo aleatório acima de um negativo aleatório

Interpretação Probabilística (resumida)

$$AUC = P(\hat{p}_{\text{spam}} > \hat{p}_{\text{legítimo}})$$

AUC = 0.88 significa: em 88% dos pares (spam, legítimo) sorteados aleatoriamente, o modelo atribui probabilidade *maior* ao spam, mesmo sem definir nenhum limiar.

⚠ Limitações da AUC-ROC

- ▶ **Classes muito desbalanceadas:** muitos negativos inflam TFP e mascaram desempenho real → prefira **AUC-PR**
- ▶ **Não informa o limiar ideal:** é necessária análise adicional (curva, custo de erro) para escolher o ponto de operação
- ▶ **Não é suficiente sozinha:** AUC alta não garante bom desempenho em um limiar específico relevante ao negócio

Validação Cruzada (k-fold)

Como funciona? ($k=5$)

1. Divide os dados em k partes (*folds*)
2. Em cada rodada: usa $k - 1$ folds para treino, 1 para validação
3. Repete k vezes (cada fold serve como validação uma vez)
4. Resultado: média e desvio-padrão das k avaliações

Variante: *Stratified k-fold* mantém proporção de classes em cada fold, essencial para dados desbalanceados.

Fold 5	Tr	Tr	Tr	Tr	Val
Fold 4	Tr	Tr	Tr	Val	Tr
Fold 3	Tr	Tr	Val	Tr	Tr
Fold 2	Tr	Val	Tr	Tr	Tr
Fold 1	Val	Tr	Tr	Tr	Tr

Vantagem

Usa **todos os dados** para treino e validação.

Referências

- [1] Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81–106.
- [2] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- [3] Vapnik, V. N. (1995). *The Nature of Statistical Learning Theory*. Springer.
- [4] Chen, T., & Guestrin, C. (2016). XGBoost. *KDD 2016*, 785–794.
- [5] Ke, G., et al. (2017). LightGBM. *NeurIPS 2017*, 3149–3157.

Consulte o Material Didático

Módulo 3: Técnicas de Classificação

Exemplos práticos, exercícios resolvidos e implementações em Python disponíveis no
Notebook Colab.

 github.com/eltonsarmanho

 eltonss@ufpa.br